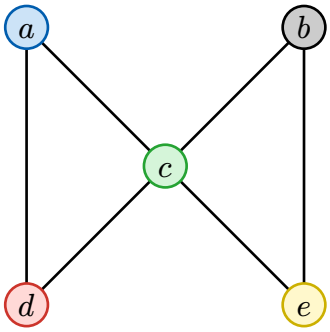
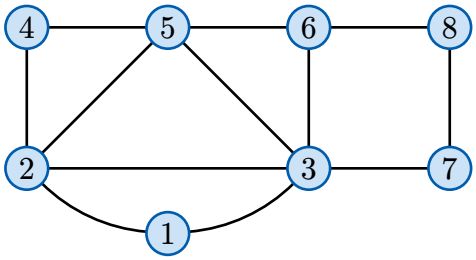
«The origins of graph theory are humble, even frivolous.» — *Норман Линстед Биггс*

1. Анализ инвариантов

(a)



(b)



a) Способы задания

- Задайте данные графы всеми известными способами

b) Основные характеристики связности

Для каждого из них вычислите указанные характеристики и свойства:

- Минимальная степень вершины $\delta(G)$, максимальная степень вершины $\Delta(G)$
- Вершинная связность $\kappa(G)$, рёберная связность $\lambda(G)$
- Все $(\kappa - 1)$-connected компоненты, $(\lambda + 1)$-edge-connected компоненты
- Проверка неравенства Уитни: $\kappa(G) \leq \lambda(G) \leq \delta(G)$

c) Простой граф

- Докажите, что в каждом простом графе $(n - 1)$ найдутся вершины $u \neq v : \deg(u) = \deg(v)$
- При каких n существуют простые графы с n вершинами каждая из которых имеет степень 3
- Какое наименьшее и наибольшее число ребер может быть в графе с n вершинами и k компонентами связности?

2. Степени, последовательности

Последовательность $d = (d_1, d_2, ..., d_n)$, где $d_1 \geq d_2 \geq ... \geq d_n \geq 0$ называется *графической*, если существует простой граф с этой последовательностью степеней.

Для каждой последовательности определите, является ли она графической.

- Если да - постройте реализующий граф (используя алгоритм Хавела-Хакими).
- Если нет - объясните почему (используя критерий Эрдёша-Галлаи).

(a) $(5, 4, 3, 2, 2, 2)$
(b) $(3, 3, 3, 3, 3, 3)$
(c) $(4, 3, 2, 1, 0)$

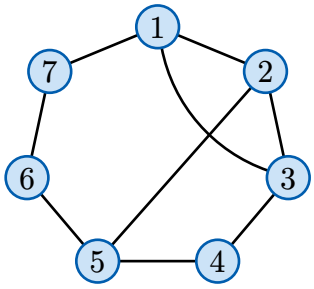
(d) $(6, 3, 3, 3, 3, 2, 2)$
(e) $(1, 1, 1, 1, 1, 1)$

3. Изоморфизм графов

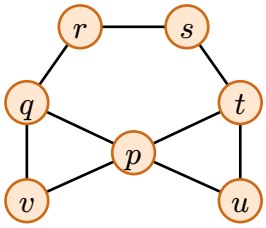
a) Математическая основа

- Для каждой изоморфной пары графов укажите явную биекцию: $f : V(G) \to V(H)$ сохраняющую смежность.
- Для неизоморфных укажите различающий инвариант (например: последовательность степеней, обхват и тд).

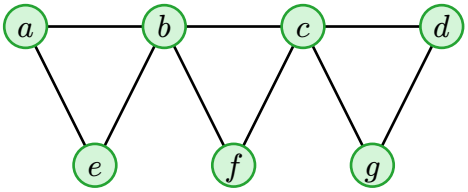
Graph H_1:



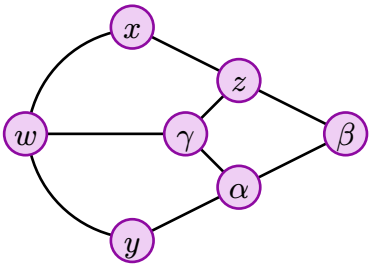
Graph H_2:



Graph H_3:

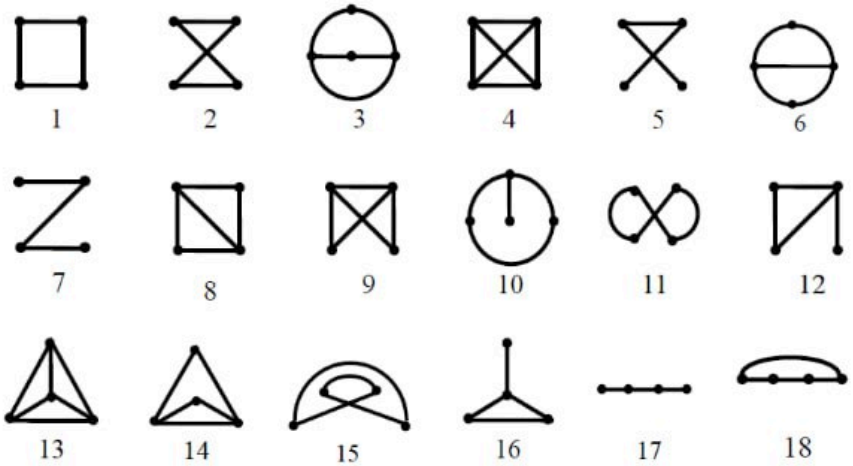


Graph H_4:



b) Классы эквивалентности

Разбить данное множество графов на классы эквивалентности по отношению изоморфности графов:



Предложить инварианты (или системы инвариантов), разделяющие данные классы.

4. Транзакционность

Чтобы понять, не «сломают» ли друг другу программу одновременно работающие куски кода, нужно посмотреть, в каком порядке они обращаются к переменным. Мы должны построить схему (хронологию) того, кто и когда читал или менял данные. Чтобы код работал предсказуемо и корректно, нам нужно жестко договориться, кто и когда имеет право читать или писать данные. Нужно запретить хаос и установить четкую очередь (строгую последовательность) на операции чтения и записи. Чтобы проверить, нет ли в нашем расписании логической ошибки используется граф предшествования[1]

Граф предшествования - это ориентированный граф, где:
- вершины представляют инструкции программы
- рёбра представляют зависимости между инструкциями

Постройте граф предшествования для программы:

```
S1: x := 0
S2: x := x + 1
S3: y := 2
S4: z := y
S5: x := x + 2
S6: y := x + z
S7: z := 4
```

[1]На самом деле все что мы описали используется в моделях конфликтов транзакций с которыми вы познакомитесь на 2 курсе